

Module 7

Secondary Storage

Secondary Storage – Magnetic Disk

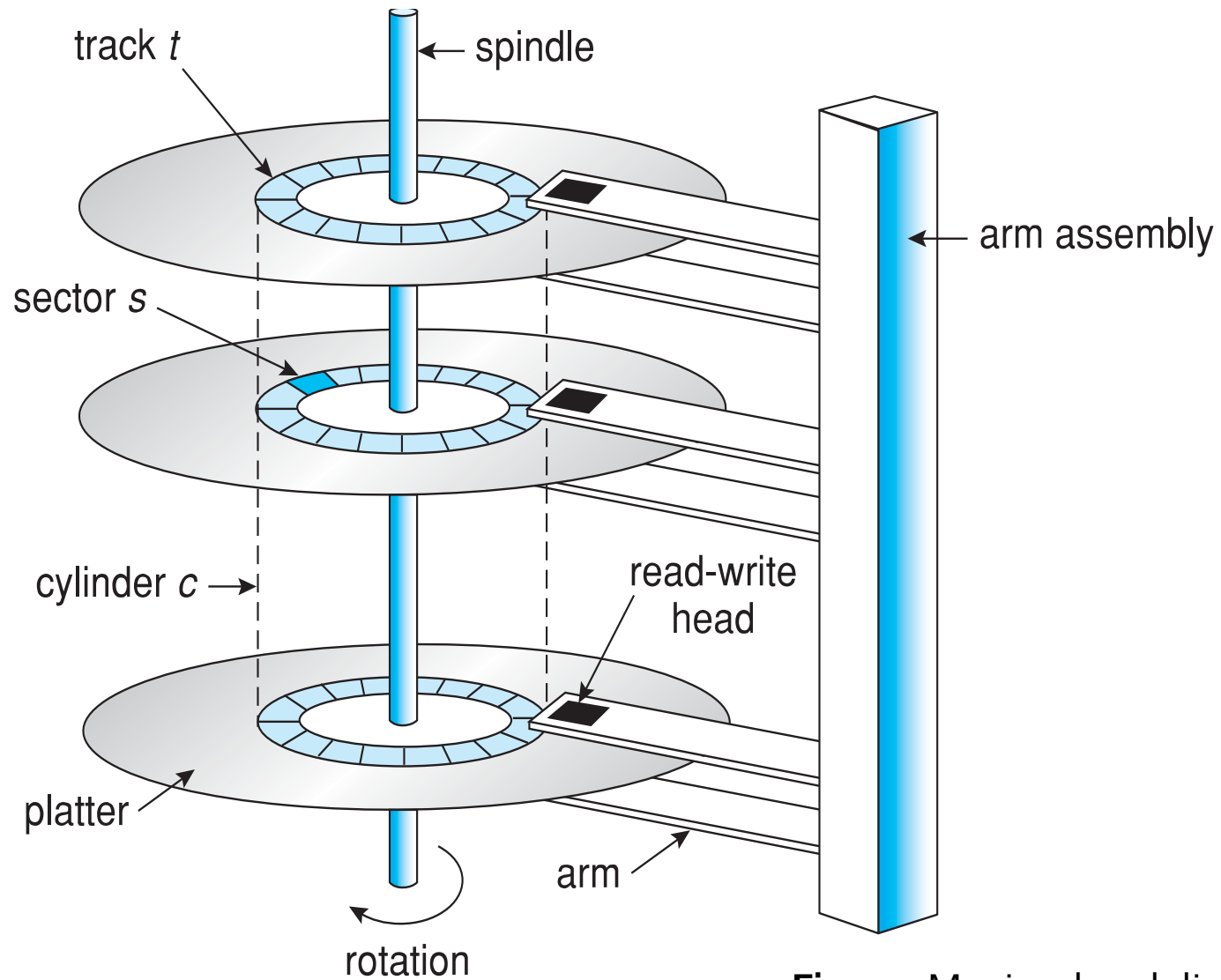


Figure: Moving-head disk mechanism of magnetic disk.

Magnetic Disk

Each disk **platter** has a flat circular shape with a diameters range from 1.8 to 3.5 inches.

The surface of a platter is logically divided into circular **tracks**, which are subdivided into **sectors**. The set of tracks that are at one arm position makes up a **cylinder**.

Most drives rotate 60 to 250 times per second, specified in terms of rotations per minute (**RPM**).

A disk drive is attached to a computer by a set of wires called an **I/O Bus** (advanced technology attachment (ATA), serial ATA (SATA), eSATA, universal serial bus (USB), and fibre channel (FC)

Magnetic Disk

Disk speed has two parts.

The **transfer rate** is the rate at which data flow between the drive and the computer.

The **positioning time**, or **random-access time**, consists of two parts:

- the time necessary to move the disk arm to the desired cylinder, called the **seek time**, and
- the time necessary for the desired sector to rotate to the disk head, called the **rotational latency**.

Typical disks can transfer several megabytes of data per second, and they have seek times and rotational latencies of several milliseconds.

Solid State Drive

- Nonvolatile memory used like a hard drive
- Many technology variations
- Can be more reliable than HDDs
- More expensive per MB
- Maybe have shorter life span
- Less capacity
- But much faster
- Busses can be too slow -> connect directly to PCI for example
- No moving parts, so no seek time or rotational latency

Magnetic Tapes

- Was early secondary-storage medium
- Evolved from open spools to cartridges
- Relatively permanent and holds large quantities of data
- Access time slow
- Random access ~1000 times slower than disk
- Mainly used for backup, storage of infrequently-used data, transfer medium between systems
- Kept in spool and wound or rewound past read-write head
- Once data under head, transfer rates comparable to disk
- 140MB/sec and greater
- 200GB to 1.5TB typical storage
- Common technologies are LTO- $\{3,4,5\}$ and T10000

Disk Structure

- Disk drives are addressed as large 1-dimensional arrays of logical blocks, where the logical block is the smallest unit of transfer
- The size of a logical block is usually 512 bytes or 1024 bytes
- Low-level formatting creates logical blocks on physical media
- The 1-dimensional array of logical blocks is mapped into the sectors of the disk sequentially
- Sector 0 is the first sector of the first track on the outermost cylinder
- Mapping proceeds in order through that track, then the rest of the tracks in that cylinder, and then through the rest of the cylinders from outermost to innermost.

Disk Structure

- By using this mapping, we can convert a **logical block number into disk address** that consists of a *cylinder number*, a *track number* within that cylinder, and *a sector number* within that track
- Mapping is difficult to perform for two reasons:
 - First, **most disks have some defective sectors**, but the mapping hides this by substituting spare sectors from elsewhere on the disk.
 - Second, the **number of sectors per track is not a constant** on some drives. i.e. On media that use **constant linear velocity (CLV)**, the density of bits per track is uniform. Tracks in the outermost zone typically hold 40 percent more sectors than do tracks in the innermost zone. The drive increases its rotation speed so that the head keeps the same rate of data.

Disk Structure

Alternatively, the disk rotation speed can stay constant; in this case, the density of bits decreases from inner tracks to outer tracks to keep the data rate constant.

This method is used in hard disks and is known as **constant angular velocity** (CAV).

Disk Attachment

Computers access disk storage in **two ways**:

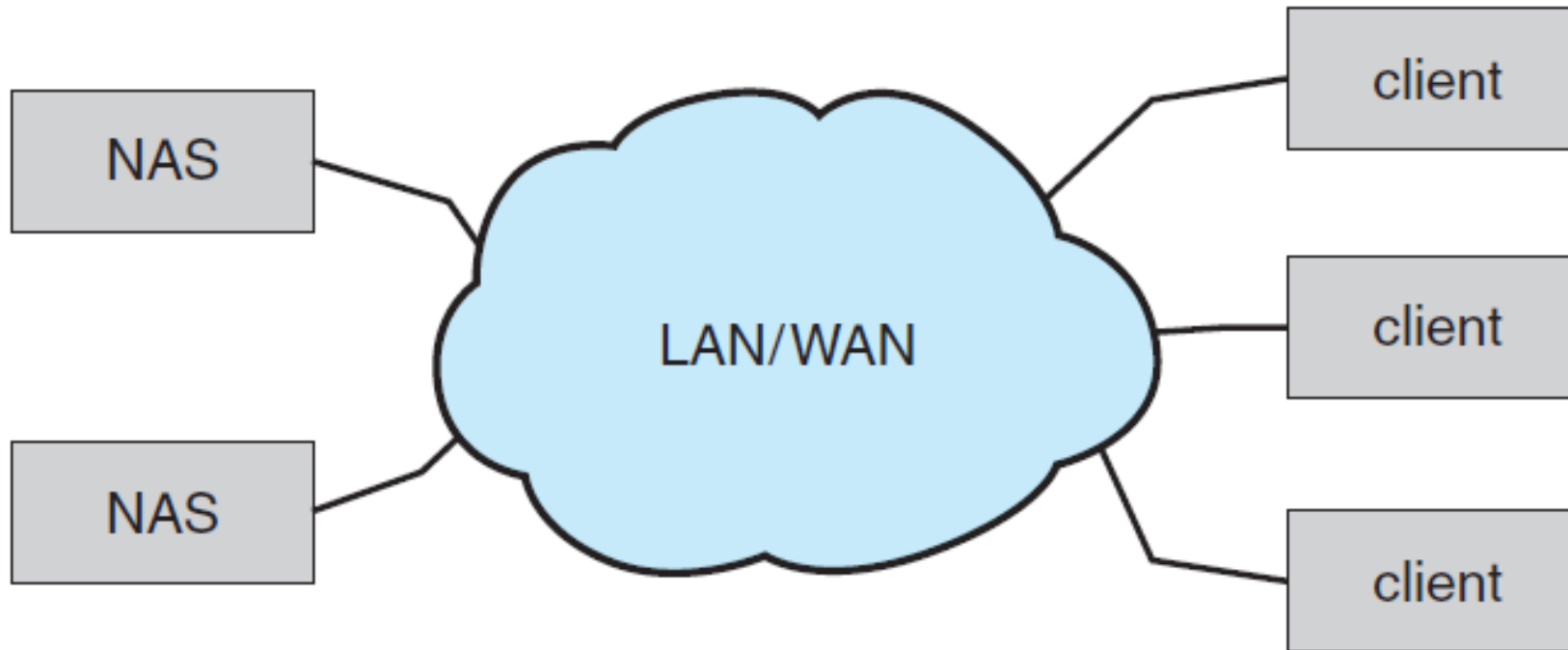
One way is via I/O ports (or **host-attached storage**);

The other way is via a remote host in a distributed file system; this is referred to as **network-attached storage**.

Host-attached storage

- Host-attached storage is storage accessed through local I/O ports.
- A typical desktop PC uses an I/O bus architecture called IDE or ATA. This architecture supports a maximum of two drives per I/O bus.
- High-end workstations and servers generally use more sophisticated I/O architectures such as Fibre Channel (FC), a high-speed serial architecture that can operate over optical fiber.
- Storage devices such as hard disk drives, RAID arrays, and CD, DVD, and tape drives are suitable for use as host-attached storage.
- The I/O commands that initiate data transfers to a host-attached storage device are reads and writes of logical data blocks

Network-Attached Storage (NAS)



Network-Attached Storage (NAS)

A network-attached storage (NAS) device is a **special-purpose storage system** that is **accessed remotely over a data network**

Clients access network-attached storage via a **remote-procedure-call interface** such as NFS for UNIX systems or CIFS for Windows machines.

The remote procedure calls (RPCs) are carried via **TCP or UDP over an IP network**

The network attached storage unit is implemented as a **RAID array** with software that implements the RPC interface.

iSCSI is the latest network-attached storage protocol. In essence, it uses the IP network protocol to carry the SCSI protocol.

Storage Area Network

*One drawback of network-attached storage systems is that the storage I/O operations **consume bandwidth** on the data network, **thereby increasing the latency** of network communication.*

*Second drawback is the **single point of failure***

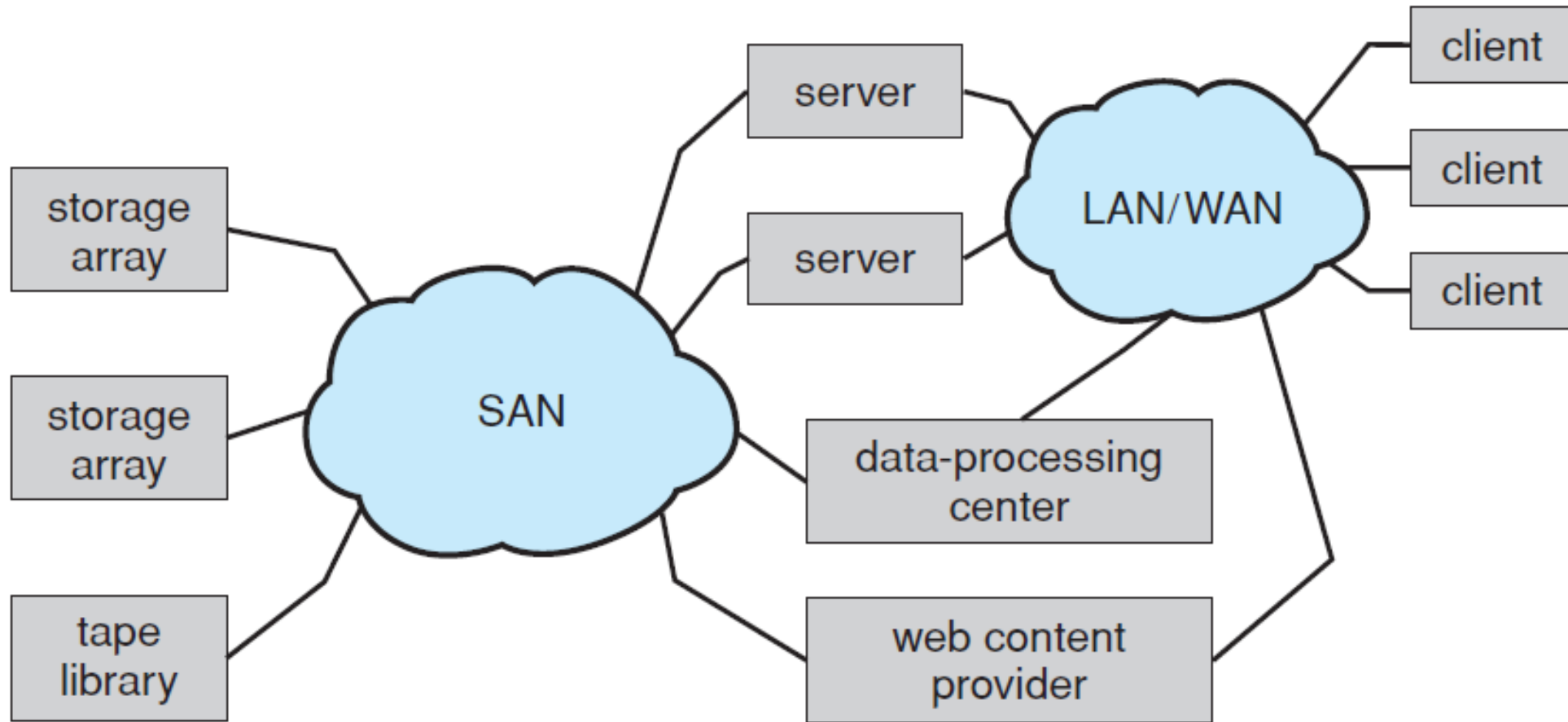
*A **Storage-Area Network (SAN)** is a private network (using storage protocols rather than networking protocols) connecting servers and storage units.*

It is very flexible.

*Multiple hosts and multiple storage **arrays can attach to the same SAN**, and storage can be **dynamically allocated** to hosts. A SAN switch allows or prohibits access between the hosts and the storage.*

SAN interconnect – Fibre channel, iSCSI, Infiniband

Storage Area Network



Disk Scheduling

For magnetic disks, the access time has two major components:

- The **seek time** is the time for the disk arm to move the heads to the cylinder containing the desired sector.
- The **rotational latency** is the additional time for the disk to rotate the desired sector to the disk head.

The **disk bandwidth** is the *total number of bytes transferred, divided by the total time between the first request for service and the completion of the last transfer.*

Access time and the bandwidth can be improved by managing the order in which disk I/O requests are serviced.

Disk Scheduling

Disk Access Time = Seek Time + Rotational Latency + Transfer Time

Total Seek Time = Total Head Movement * Seek Time

Disk Scheduling

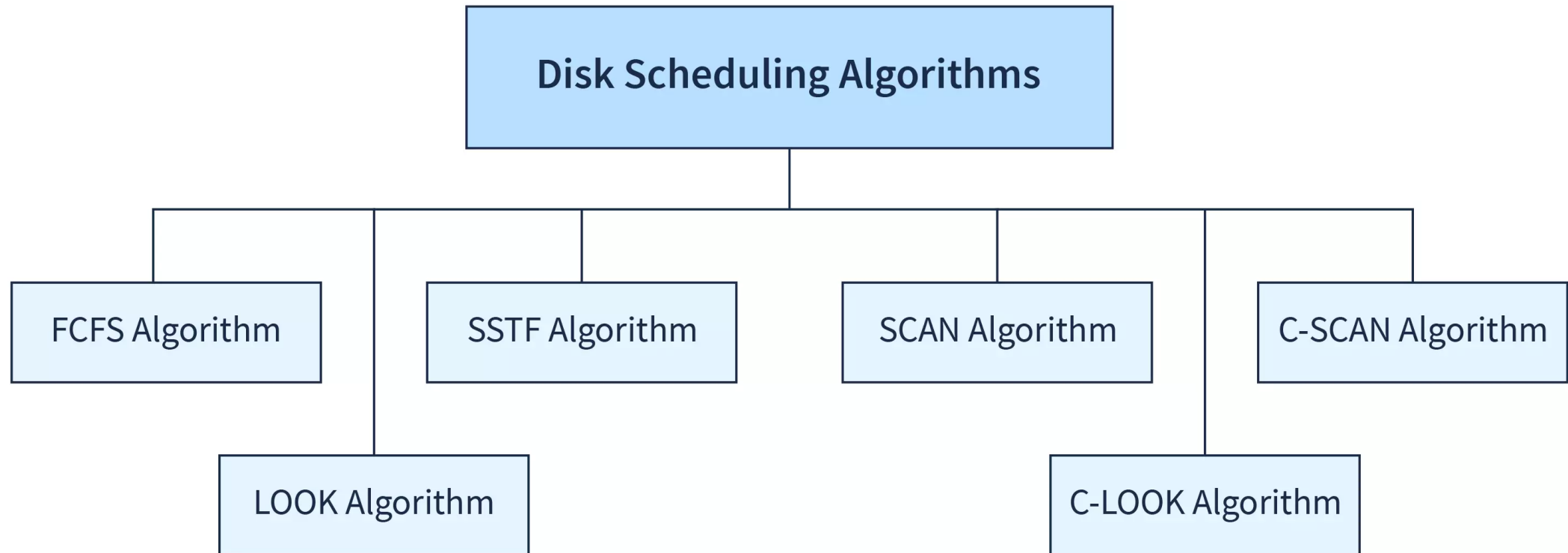
Whenever a process needs I/O to or from the disk, it issues a system call to the operating system. The request specifies several pieces of information:

- *Whether this operation is input or output*
- *What the disk address for the transfer is*
- *What the memory address for the transfer is*
- *What the number of sectors to be transferred is*

If the desired disk drive and controller are available, the request can be serviced immediately. If the drive or controller is busy, any new requests for service will be placed in the queue of pending requests for that drive.

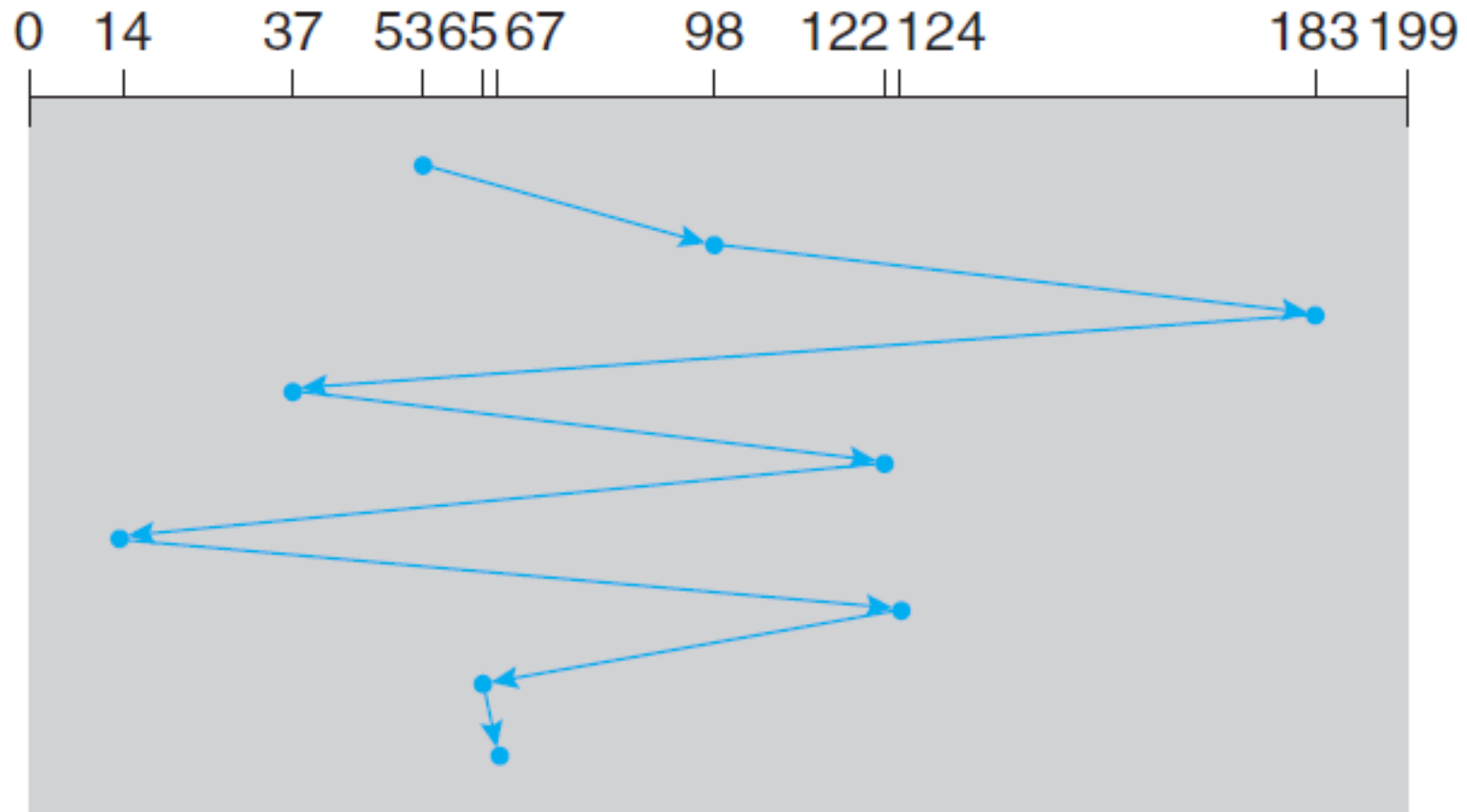
Operating system chooses the pending request to service next using disk scheduling algorithm

Disk Scheduling Algorithms



1. FCFS Scheduling

Consider a disk queue with requests for I/O to blocks on cylinders:
98, 183, 37, 122, 14, 124, 65, 67 and head starts at 53.



1. FCFS Scheduling

Consider a disk queue with requests for I/O to blocks on cylinders:
98, 183, 37, 122, 14, 124, 65, 67 and head starts at 53.

Total head movement of 640 cylinders

This algorithm is fair, but it generally does not provide the fastest service

It is simple to implement.

It does not cause starvation to any request.

It suffers from increased total seek time

2. SSTF Scheduling

The shortest-seek-time-first (SSTF) algorithm selects the request with the least seek time from the current head position.

“Service all the requests close to the current head position before moving the head far away to service other requests”

SSTF chooses the pending request closest to the current head position.

2. SSTF Scheduling

Advantage:

Improves performance,

As compared to FCFS, the total seek time is low.

The average response time is very low due to the fewer head movements.

Waiting time is low.

Disadvantage:

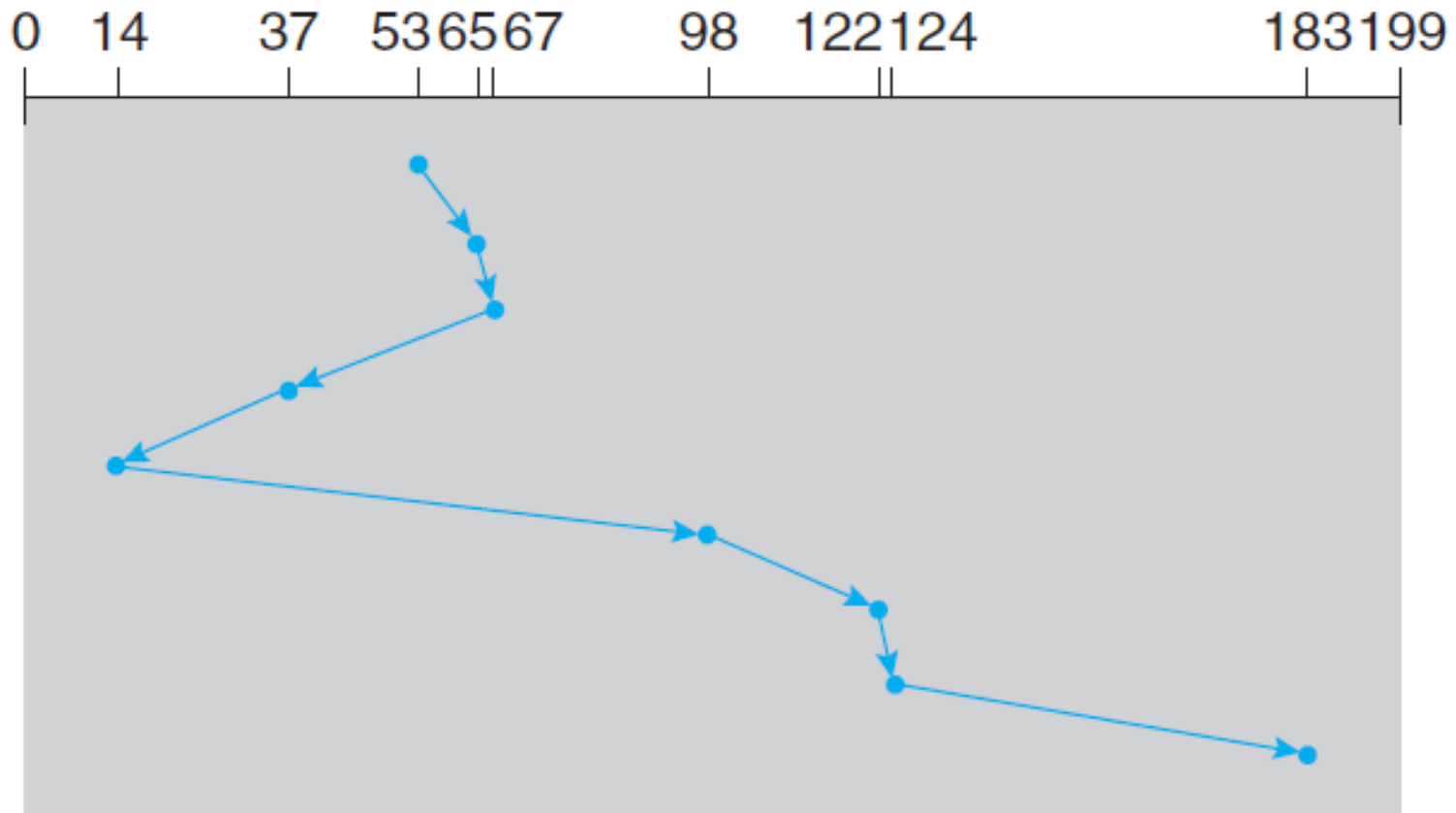
Overhead is high because SSTF calculates the seek time of every request in advance.

It causes starvation of a request if it has a higher seek time.

Requests with a lower seek time in a queue get a higher priority even if they didn't arrive first- making the algorithm unfair.

2. SSTF Scheduling

Consider a disk queue with requests for I/O to blocks on cylinders:
98, 183, 37, 122, 14, 124, 65, 67 and head starts at 53.



Total head movement of only 236 cylinders

3. SCAN Scheduling

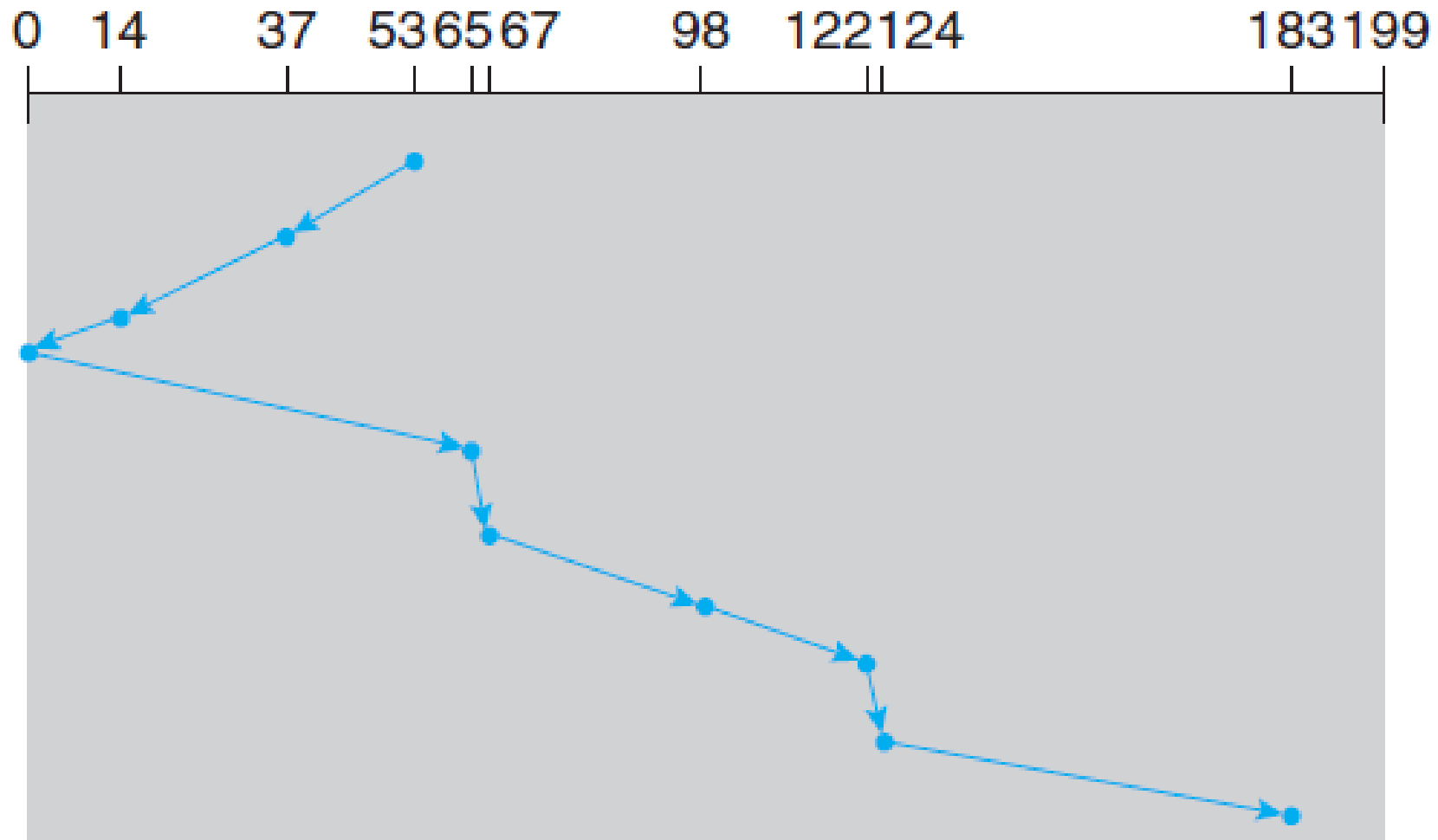
In the SCAN algorithm, the disk arm starts at one end of the disk and moves toward the other end, servicing requests as it reaches each cylinder, until it gets to the other end of the disk.

At the other end, the direction of head movement is reversed, and servicing continues.

The SCAN algorithm is sometimes called the **elevator algorithm**.

The direction of head movement is required in addition to the head's current position.

3. SCAN Scheduling



3. SCAN Scheduling

Advantages:

Implementation is easy.

Requests do not have to wait in a queue.

Disadvantage:

The head keeps going on to the end even if there are no requests in that direction.

4. C-SCAN Scheduling

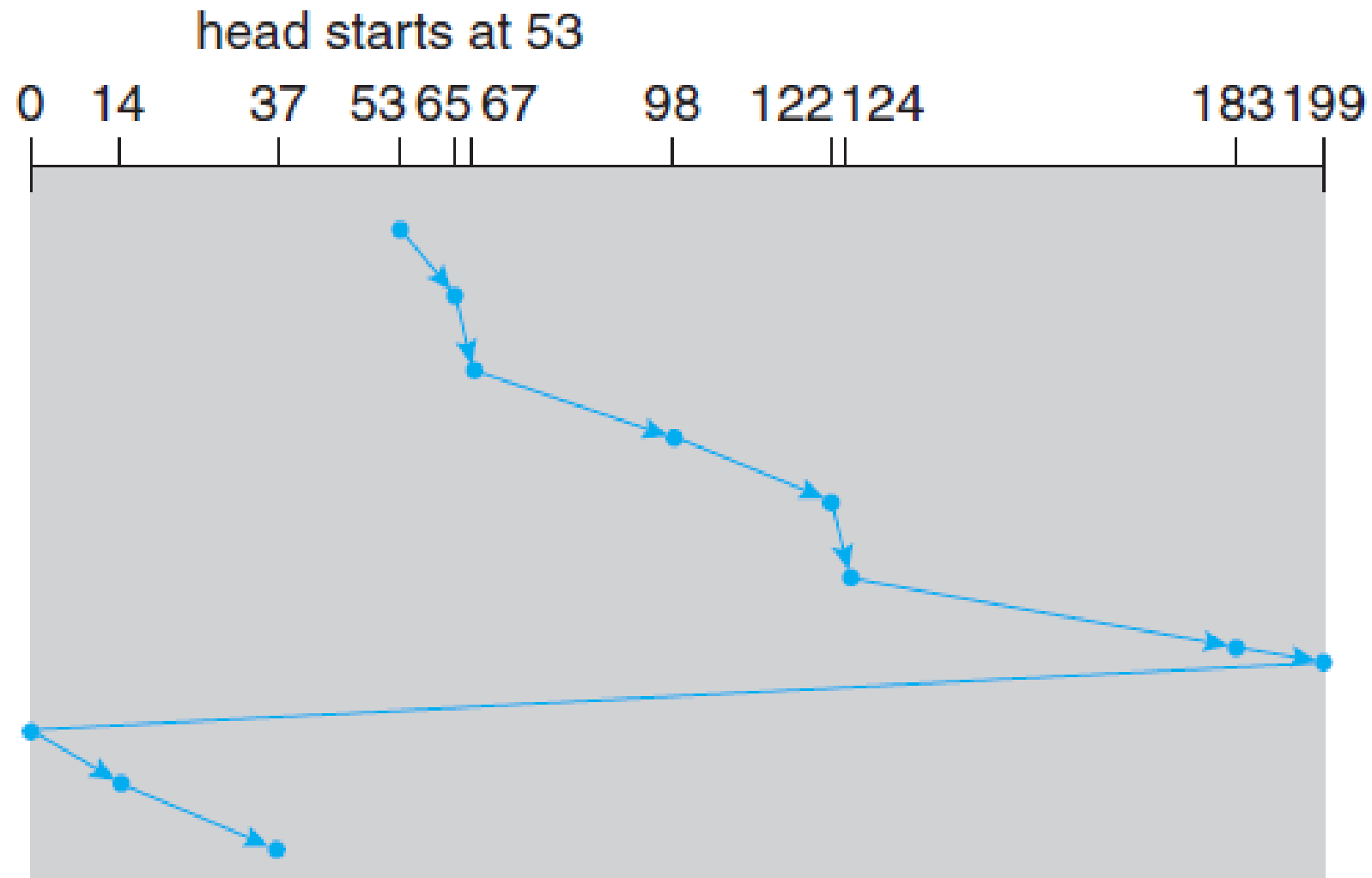
Circular SCAN (C-SCAN) scheduling is a variant of SCAN designed to provide a more uniform wait time.

Like SCAN, C-SCAN moves the head from one end of the disk to the other, servicing requests along the way.

When the head reaches the other end, however, it immediately returns to the beginning of the disk without servicing any requests on the return trip.

The C-SCAN scheduling algorithm essentially treats the cylinders as a circular list that wraps around from the final cylinder to the first one.

4. C-SCAN Scheduling



4. C-SCAN Scheduling

Advantages:

The waiting time is uniformly distributed among the requests.
Response time is good in it.

Disadvantages:

The time taken by the disk arm to locate a spot is increased here.
The head keeps going to the end of the disk.

5. LOOK Scheduling

The LOOK algorithm is similar to the SCAN algorithm but stops servicing requests as soon as it reaches the end of the disk.

This algorithm can reduce the total distance the disk head must travel, improving disk access time.

However, this algorithm can lead to long wait times for requests that are made near the end of the disk, as they must wait for the disk head to wrap around to the other end of the disk before they can be serviced.

The LOOK algorithm is often used in modern operating systems due to its ability to reduce disk access time and improve overall system performance.

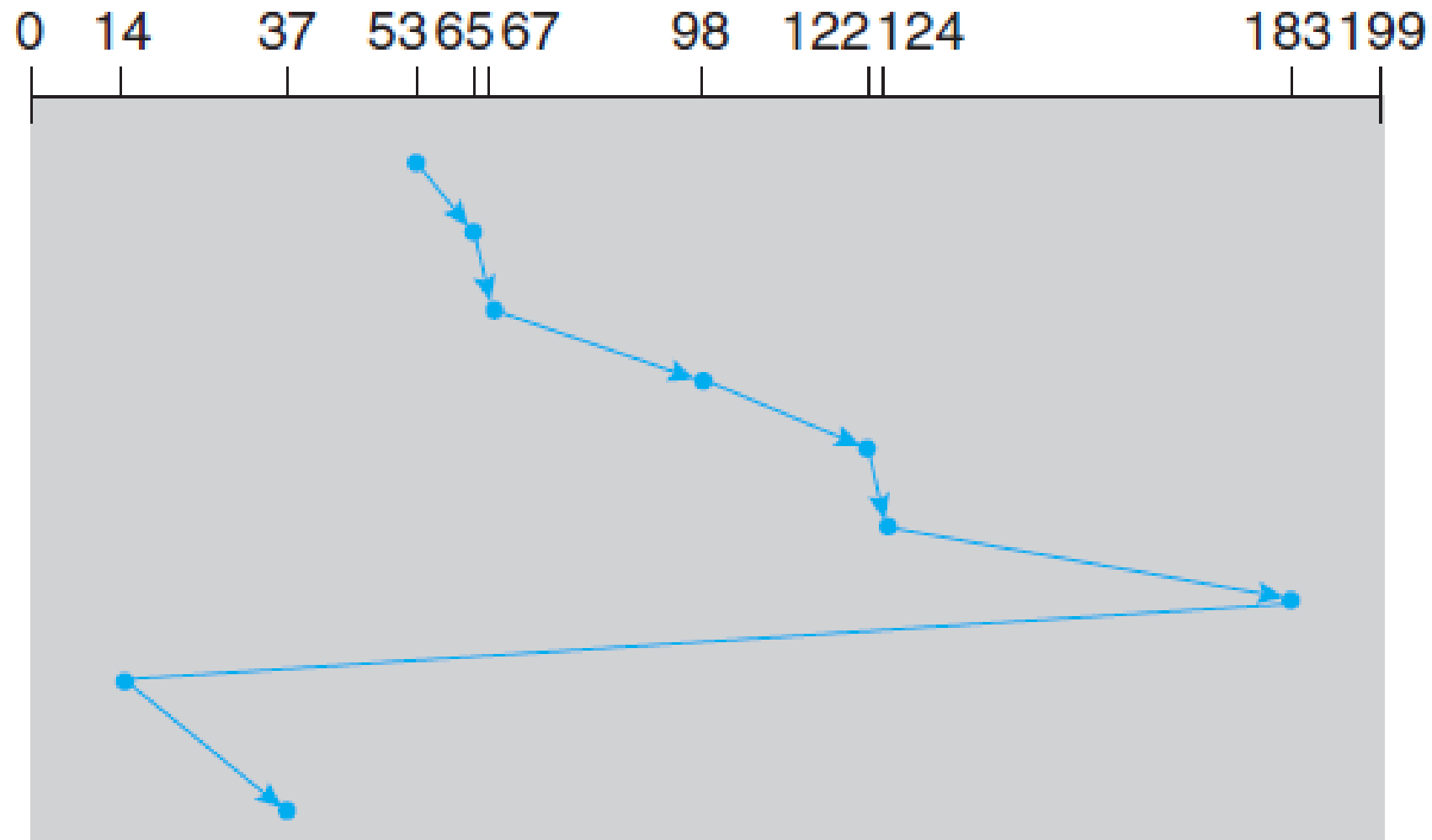
6. C-LOOK Scheduling

C-LOOK is similar to the C-SCAN disk scheduling algorithm.

In this algorithm, goes only to the last request to be serviced in front of the head in spite of the disc arm going to the end, and then from there it goes to the other end's last request.

Thus, it also prevents the extra delay which might occur due to unnecessary traversal to the end of the disk.

6. C-LOOK Scheduling



6. C-LOOK Scheduling

C-LOOK is similar to the C-SCAN disk scheduling algorithm.

In this algorithm, goes only to the last request to be serviced in front of the head in spite of the disc arm going to the end, and then from there it goes to the other end's last request.

Thus, it also prevents the extra delay which might occur due to unnecessary traversal to the end of the disk.

6. C-LOOK Scheduling

Advantages:

The waiting time is decreased.

If there are no requests till the end, it reverses the head direction immediately.

Starvation does not occur.

The time taken by the disk arm to find the desired spot is less.

Disadvantage:

The arm has to be conscious about finding the last request.

Sample Problem

Suppose the order of disk requests is (82, 170, 43, 140, 24, 16, 190). The current position of the read/write head is at 50. Find the optimal algorithm that requires less total overhead movement.